

EXHIBIT 20

TPU7x (Ironwood)

This page describes the architecture and available configurations for TPU7x, the latest TPU available on Google Cloud. TPU7x is the first release within the Ironwood family, Google Cloud's seventh generation TPU. The Ironwood generation is designed for large-scale AI training and inference.

With a 9,216-chip footprint per Pod, TPU7x shares many similarities with [TPU v5p \(/tpu/docs/v5p\)](#). TPU7x provides high performance for large scale dense and MoE models, pre-training, sampling and decode-heavy inference.

To use TPU7x, you can use Google Kubernetes Engine (GKE) or Compute Engine. For more information about using TPUs with GKE, see [About TPUs in GKE \(/kubernetes-engine/docs/concepts/tpus\)](#).

You can also use TPU7x and GKE with All Capacity mode. All Capacity mode is available through an All Capacity mode reservation, which gives you full access to all of your reserved capacity (no hold-backs) and full visibility into the TPU hardware topology, utilization status, and health status. For more information, see [All Capacity mode overview \(/tpu/docs/all-capacity-overview\)](#).

Note: You can use the JAX and PyTorch frameworks on TPU7x. TensorFlow is not supported.

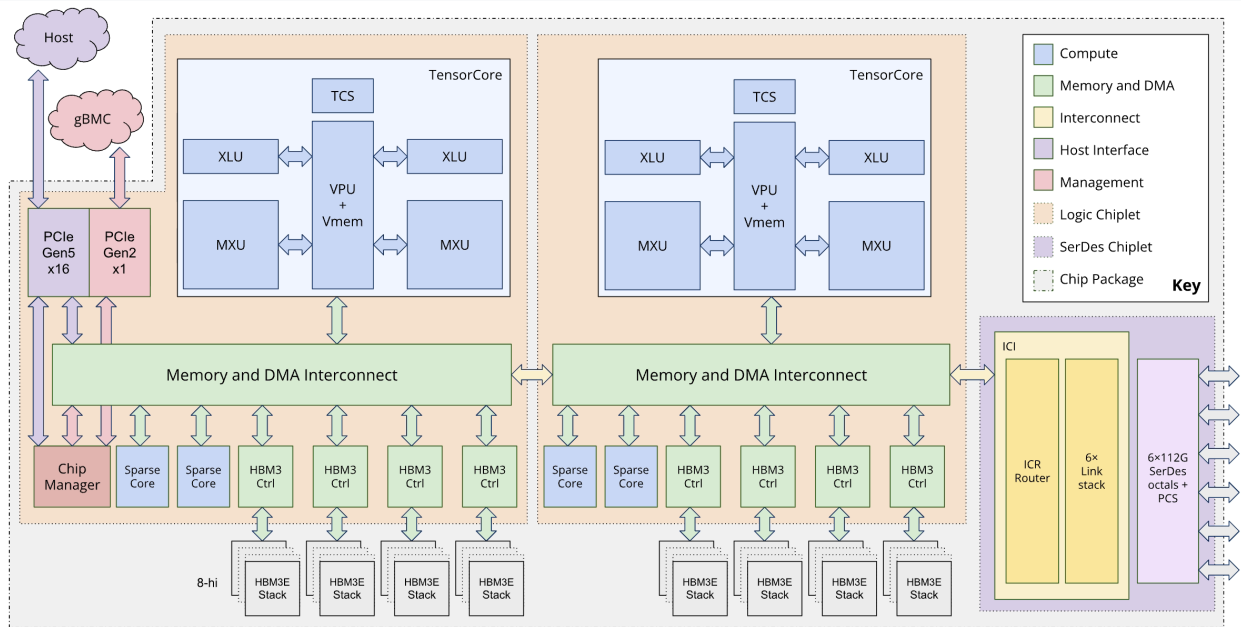
System architecture

Each TPU7x chip contains two TensorCores and four SparseCores. The following table shows the key specifications and their values for TPU7x compared to prior generations.

Specification	v5p	v6e (Trillium)	TPU7x (Ironwood)
Number of chips per pod	8960	256	9216
Peak compute per chip (BF16) (TFLOPs)	459	918	2307
Peak compute per chip (FP8) (TFLOPs)	459	918	4614

Specification	v5p	v6e (Trillium)	TPU7x (Ironwood)
HBM capacity per chip (GiB)	95	32	192
HBM bandwidth per chip (GBps)	2765	1638	7380
Number of vCPUs (4-chip VM)	208	180	224
RAM (GB) (4-chip VM)	448	720	960
Number of TensorCores per chip	2	1	2
Number of SparseCores per chip	4	2	4
Bidirectional inter-chip interconnect (ICI) bandwidth per chip (GBps)	1200	800	1200
Data center network (DCN) bandwidth per chip (Gbps)	50	100	100

The following diagram illustrates the architecture of Ironwood:



Memory hierarchy

TPU7x features a multi-tiered memory system, and managing data movement between these tiers is crucial for performance:

- **High-bandwidth memory (HBM):** Each chip is equipped with 192 GB of HBM, with bandwidth of approximately 7.37 TB/s. The large HBM capacity enables large batch

sizes, which can improve throughput. However, despite its size, HBM can still be a bottleneck, particularly for memory-bound vector operations or inefficient data access patterns.

- **Vector memory (VMEM):** VMEM is a smaller, on-chip SRAM (static random-access memory) with significantly higher bandwidth to the Matrix Multiply Unit (MXU) than HBM. This memory acts as a high-speed scratchpad for custom kernels. The size of this buffer is a tunable parameter. Optimizing the buffer size is critical for tuning custom Pallas kernels, as their block sizes are often constrained by the available VMEM.
- **Host memory and PCIe:** Each set of four TPU chips is connected to a CPU host using a PCIe network. While this connection has much lower bandwidth than HBM, the host's main memory can be used for offloading activations or optimizer states to free up HBM, a technique particularly useful for managing memory pressure in large models.

For more on efficiently managing data movement between the tiers of the TPU7x memory hierarchy, see [Ironwood performance optimizations](/tpu/docs/ironwood-performance) (/tpu/docs/ironwood-performance).

Dual-chiplet architecture

The Ironwood programming model lets you access two TPU chiplets instead of a single logical core (also known as [MegaCore](#) (<https://docs.jax.dev/en/latest/pallas/tpu/pipelining.html#tpus-in-megacore-configuration>)) architecture used in previous generations (TPU v4 and v5p). This change improves the cost-effectiveness and efficiency of manufacturing the chip. While this represents an architectural shift, the new design ensures that you can reuse existing software models with minimal changes.

Ironwood TPUs are composed of two distinct chiplets, each with its own dedicated memory space. This is a departure from the unified memory space of the MegaCore architecture.

- **Chiplet composition:** Each chiplet is a self-contained unit with one TensorCore, two SparseCores, and 96 GB of high-bandwidth memory (HBM).
- **High-speed interconnect:** The two chiplets are connected by a die-to-die (D2D) interface that is six times faster than a 1D inter-chip interconnect (ICI) link. Inter-chiplet communication is managed using collective operations.

Programming model and framework exposure

The programming model for Ironwood is similar to that of TPU generations earlier than v4, such as TPU v3. The new architecture is exposed in the following ways:

- **Two devices per chip:** Frameworks like JAX expose each Ironwood chip as two separate "devices," one for each chiplet.
- **Chiplet specification:** You can specify which chiplet to use for a computation. JAX adds a fourth dimension to the topology specification to distinguish between chiplets. This design lets you reuse existing software models with minimal changes.

For more information about achieving optimal performance with the dual-chiplet architecture, see [Performance recommendations for Ironwood's dual-chiplet architecture](#) (/tpu/docs/performance-guide#ironwood-dual-chiplet)

Supported configurations

Note: Use the [TPU topology visualizer](https://tpu-visualizer.uc.r.appspot.com/) (https://tpu-visualizer.uc.r.appspot.com/) to view 3D renderings of different TPU configurations.

TPU7x chips have a direct connection to the nearest neighboring chips in 3 dimensions, resulting in a 3D mesh of networking connections. Slices larger than 64 chips are made up of one or more 4x4x4 "cubes" of chips.

TPU7x chips have a 3D torus interconnect topology. This topology allows slices to scale up to 9216 chips. It has bi-directional bandwidth of 200 GBps per axis for communication between chips within a pod.

The following table shows common 3D slice shapes that are supported for TPU7x:

Topology	TPU chips	Hosts	VMs	Cubes	Scope
2x2x1	4	1	1	1/16	Single-host
2x2x2	8	2	2	1/8	Multi-host
2x2x4	16	4	4	1/4	Multi-host
2x4x4	32	8	8	1/2	Multi-host

4x4x4	64	16	16	1	Multi-host
4x4x8	128	32	32	2	Multi-host
4x8x8	256	64	64	4	Multi-host
8x8x8	512	128	128	8	Multi-host
8x8x16	1024	256	256	16	Multi-host
8x16x16	2048	512	512	32	Multi-host

Note: All TPU7x topologies use the **tpu7x-standard-4t** machine type.

TPU7x VM

Each TPU7x virtual machine (VM) contains 4 chips. Each VM has access to two NUMA nodes. For more information about NUMA nodes, see [Non-uniform memory access](https://en.wikipedia.org/wiki/Non-uniform_memory_access) (https://en.wikipedia.org/wiki/Non-uniform_memory_access) on Wikipedia.

All TPU7x slices use full-host, 4-chip VMs. The technical specifications for a TPU7x VM are:

- **Number of vCPUs per VM:** 224
- **RAM per VM:** 960 GB
- **Number of NUMA nodes per VM:** 2

Hyperdisk

By default, the VM boot disk for TPU7x is Hyperdisk Balanced. You can attach more disks to your TPU VM for additional storage. The following disk types are supported on TPU7x:

- Hyperdisk Balanced
- Hyperdisk ML

For more information about Hyperdisk, see [Hyperdisk overview](/compute/docs/disks/hyperdisks) (/compute/docs/disks/hyperdisks). For more information about storage options for Cloud TPU, see [Storage options for Cloud TPU data](/tpu/docs/storage-options-compute) (/tpu/docs/storage-options-compute).

What's next

- [Use TPU7x with GKE](/kubernetes-engine/docs/concepts/tpus) (/kubernetes-engine/docs/concepts/tpus)
- [Use TPU7x with All Capacity mode](/tpu/docs/all-capacity-overview) (/tpu/docs/all-capacity-overview)
- [Use the Google Cloud ML Diagnostics platform to optimize and diagnose your workloads](https://github.com/AI-Hypercomputer/google-cloud-ml-diagnostics) (https://github.com/AI-Hypercomputer/google-cloud-ml-diagnostics)
- [Run a training workload using a recipe optimized for TPU7x](https://github.com/AI-Hypercomputer/tpu-recipes/blob/main/training/ironwood/README.md) (https://github.com/AI-Hypercomputer/tpu-recipes/blob/main/training/ironwood/README.md)
- [Run a TPU7x microbenchmark](https://github.com/AI-Hypercomputer/accelerator-microbenchmarks/blob/main/Ironwood/Ironwood_Microbenchmarks_readme.md) (https://github.com/AI-Hypercomputer/accelerator-microbenchmarks/blob/main/Ironwood/Ironwood_Microbenchmarks_readme.md)
- [Ironwood performance optimizations](/tpu/docs/ironwood-performance) (/tpu/docs/ironwood-performance)
- [Train large-scale machine learning models on GKE with Multi-Tier Checkpointing](/kubernetes-engine/docs/how-to/machine-learning/training/multi-tier-checkpointing) (/kubernetes-engine/docs/how-to/machine-learning/training/multi-tier-checkpointing)

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-07-22 UTC.